

Homework 8: Welcome to the Data Layer!

MySQL vs DynamoDB — Shopping Cart API
Database Comparison Report

CS6650 Distributed Systems
March 21, 2026

Table of Contents

1. Infrastructure Setup & Deployment

2. STEP I: MySQL Implementation

- 2.1 Schema Design Decisions
- 2.2 API Health Check & Verification
- 2.3 Performance Testing (150 Operations)
- 2.4 CloudWatch Monitoring

3. STEP II: DynamoDB Implementation

- 3.1 Table Design & Partition Key Strategy
- 3.2 API Health Check & Verification
- 3.3 Performance Testing (150 Operations)
- 3.4 CloudWatch Monitoring

4. STEP III: Database Comparison & Analysis

- 4.1 Performance Comparison Table
- 4.2 Operation-Specific Breakdown
- 4.3 Consistency Model Impact
- 4.4 Resource Efficiency Analysis
- 4.5 Real-World Scenario Recommendations
- 4.6 Architecture Recommendations

5. Learning Reflection

1. Infrastructure Setup & Deployment

This assignment extends the HW5 horizontal scaling infrastructure (ECS Fargate on AWS) with two persistent database backends: Amazon RDS MySQL and Amazon DynamoDB. The same Go-based Shopping Cart API runs on ECS and connects to either backend via the `DB_BACKEND` environment variable.

1.1 Terraform Initialization

The Terraform configuration was extended from HW5 with two new modules: `modules/rds` (for Amazon RDS MySQL) and `modules/dynamodb` (for DynamoDB table). The existing network, ECR, ECS, and logging modules were preserved unchanged.

```
Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0"...
- Finding hashicorp/aws versions matching "~> 6.7.0"...
- Installing kreuzwerker/docker v3.6.2...
- Installed kreuzwerker/docker v3.6.2 (self-signed, key ID BD080C4571C6104C)
- Installing hashicorp/aws v6.7.0...
- Installed hashicorp/aws v6.7.0 (signed by HashiCorp)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Figure 1: terraform init — Successfully initialized with AWS and Docker providers

1.2 Docker Build

The Go Shopping Cart API was compiled into a Docker image. The application supports both MySQL and DynamoDB backends through a common `CartStore` interface, selectable via environment variables.

```
[+] Building 11.2s (18/18) FINISHED                                docker:desktop-linux
=> [builder 1/6] FROM docker.io/library/golang:1.25-alpine@sha256:8e02eb337d9e0ea459e041fee5eece41cbb61f1d83e7d883a3e2fb4862063fa 0.0s
=> => resolve docker.io/library/golang:1.25-alpine@sha256:8e02eb337d9e0ea459e041fee5eece41cbb61f1d83e7d883a3e2fb4862063fa 0.0s
=> CACHED [stage-1 1/4] FROM docker.io/library/alpine:3.19@sha256:6baf43584bcb78f2e5847d1de515f23499913ac9f12bdf834811a3145eb11ca1 0.0s
=> => resolve docker.io/library/alpine:3.19@sha256:6baf43584bcb78f2e5847d1de515f23499913ac9f12bdf834811a3145eb11ca1 0.0s
=> [internal] load build context 0.7s
=> => transferring context: 13.33MB 0.7s
=> [stage-1 2/4] RUN apk add --no-cache ca-certificates 1.0s
=> CACHED [builder 2/6] WORKDIR /app 0.0s
=> [builder 3/6] COPY go.mod go.sum ./ 0.0s
=> [builder 4/6] RUN go mod download 0.8s
=> [stage-1 3/4] WORKDIR /app 0.0s
=> [builder 5/6] COPY . . 0.1s
=> [builder 6/6] RUN CGO_ENABLED=0 GOOS=linux go build -o server . 7.9s
=> [stage-1 4/4] COPY --from=builder /app/server . 0.1s
=> exporting to image 0.7s
=> => exporting layers 0.5s
=> => exporting manifest sha256:0a1fa8f204dcbc4d5129516a0032d63fb3eb9b21f35f21efa75cbf0a4265c144 0.0s
=> => exporting config sha256:d5d9a4cf9b7f15dc6181aab1b71ff8b4dddc12f58d654a94bb4711351ca6bea 0.0s
=> => exporting attestation manifest sha256:5410d36a44d26a5c0d1e6ffeb627926c5f94e94bbc0964affd2da54592fd16ec 0.0s
=> => exporting manifest list sha256:fe739035874d6d126acb74faf3ad6640cf957f6c087219365df4a2b7a97b3eac 0.0s
=> => naming to docker.io/library/hw8-cart-api:latest 0.0s
=> => unpacking to docker.io/library/hw8-cart-api:latest 0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/pb1m2jp9qb0d65jkggrbj3e8x
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework8\src>
```

Figure 2: Docker build — Successfully built hw8-cart-api image (11.2s)

1.3 Terraform Apply

All 12 AWS resources were created in a single terraform apply: VPC security groups, DB subnet group, RDS MySQL instance (db.t3.micro), DynamoDB table (on-demand billing), ECR repository, ECS cluster, task definition, and service. The RDS instance took approximately 5 minutes to provision.

```
module.network.data.aws_vpc.default: Reading...
module.rds.aws_db_instance.mysql: Still creating... [03m10s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [03m20s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [03m30s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [03m40s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [03m50s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [04m00s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [04m10s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [04m20s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [04m30s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [04m40s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [04m50s elapsed]
module.rds.aws_db_instance.mysql: Still creating... [05m00s elapsed]
module.rds.aws_db_instance.mysql: Creation complete after 5m5s [id=db-Z6GMAF34MQVNRDMFVHYVJSTMKI]
module.ecs.aws_ecs_task_definition.this: Creating...
module.ecs.aws_ecs_task_definition.this: Creation complete after 0s [id=CS6650HW8-task]
module.ecs.aws_ecs_service.this: Creating...
module.ecs.aws_ecs_service.this: Creation complete after 1s [id=arn:aws:ecs:us-west-2:591389222803:service/CS6650HW8]

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

Outputs:

dynamodb_table = "CS6650HW8-shopping-carts"
rds_endpoint = "cs6650hw8-mysql.cyqo4hpsqcvk.us-west-2.rds.amazonaws.com:3306"
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework8\terraform>
```

Figure 3: terraform apply — 12 resources created, RDS endpoint and DynamoDB table name in outputs

1.4 ECS Service Deployment

The ECS Fargate service launched successfully with 1 running task. The deployment status shows "Success" with the CS6650HW8-task:1 task definition revision.

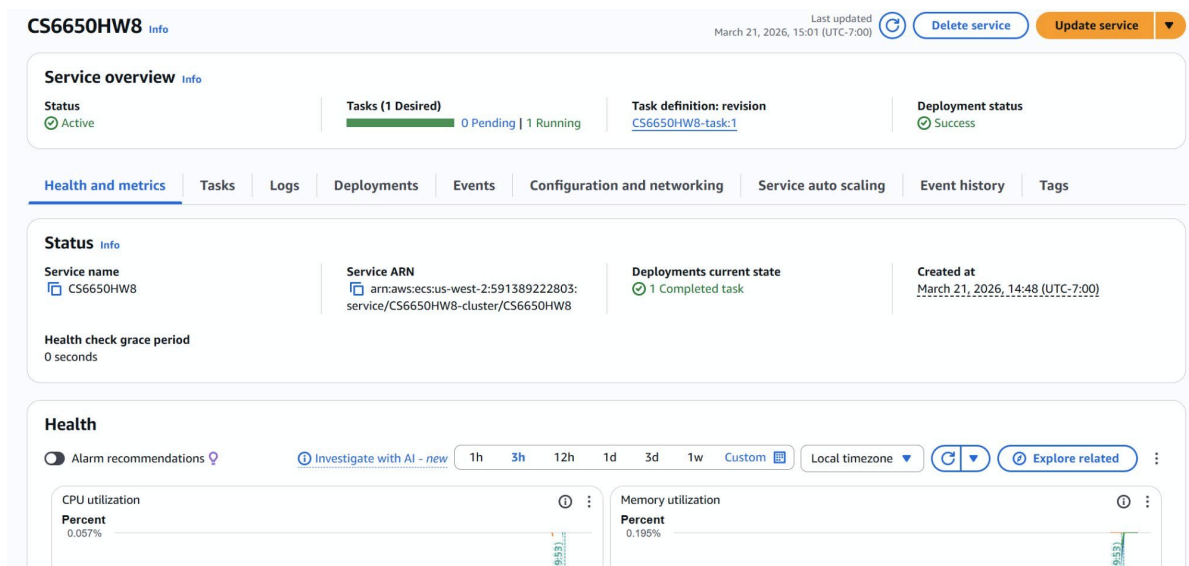


Figure 4: ECS Console — CS6650HW8 service active with 1 Running task

2. STEP I: MySQL Implementation

2.1 Schema Design Decisions

A normalized two-table schema was designed following Third Normal Form (3NF):

Table Structure: Two tables (carts + cart_items). Cart metadata is separated from item data to allow efficient queries and maintain data integrity. The trade-off is requiring a JOIN for full cart retrieval, but indexed JOINS proved fast (avg 32.28ms).

Key Strategy: Cart IDs use UUIDs (CHAR(36)) for global uniqueness across distributed ECS tasks. The cart_items table uses a compound unique constraint (cart_id, product_id) to prevent duplicate items and enable atomic upserts via INSERT ... ON DUPLICATE KEY UPDATE.

Index Strategy: Three indexes — idx_carts_customer for customer history, idx_items_cart_id for fast JOINS, and uk_cart_product (unique key) for item lookups and duplicate prevention.

Transaction Design: InnoDB row-level locking handles concurrent cart modifications. Add-item uses atomic upserts. Cart retrieval is a single read-only SELECT ... LEFT JOIN.

Connection Pool: 25 max open connections, 10 idle, 5-minute max lifetime. This balances reuse efficiency against resource consumption on the db.t3.micro instance.

2.2 API Health Check — MySQL Backend

After deployment, the health endpoint confirmed the API was running with the MySQL backend connected:

```
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content         : {"backend":"mysql","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 34
                  Content-Type: application/json
                  Date: Sat, 21 Mar 2026 22:04:41 GMT

                  {"backend":"mysql","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 34], [Content-Type, application/json], [Date, Sat, 21 Mar 2026 22:04:41 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 34
```

Figure 5: Health check — {"backend":"mysql","status":"ok"} confirming MySQL connection

2.3 Performance Testing — MySQL (150 Operations)

The required performance test was executed: 50 create_cart, 50 add_items, and 50 get_cart operations, run sequentially against the ECS-deployed API.

```
Target: http://44.242.172.77:5173
=====
[Phase 1] Creating 50 carts...
Created 10/50 (30.5ms)
Created 20/50 (30.5ms)
Created 30/50 (28.0ms)
Created 40/50 (31.5ms)
Created 50/50 (27.0ms)
-> 50 carts created successfully

[Phase 2] Adding items to 50 carts...
Added items 10/50 (33.5ms)
Added items 20/50 (35.0ms)
Added items 30/50 (36.0ms)
Added items 40/50 (32.0ms)
Added items 50/50 (30.0ms)

[Phase 3] Retrieving 50 carts...
Retrieved 10/50 (30.5ms)
Retrieved 20/50 (30.5ms)
Retrieved 30/50 (27.5ms)
Retrieved 40/50 (30.5ms)
Retrieved 50/50 (28.0ms)

=====
SUMMARY
=====
Total operations: 150
Successful:      150/150 (100.0%)
Avg response:    32.79 ms
P50:             30.50 ms
P95:             47.50 ms
P99:             51.00 ms

create_cart: avg=32.24ms min=26.00ms max=82.00ms
add_items:   avg=33.86ms min=29.51ms max=50.00ms
get_cart:    avg=32.28ms min=26.50ms max=51.00ms

Results saved to: ../mysql_test_results.json
```

Figure 6: MySQL performance test — 150/150 success, avg 32.79ms, all operations under 50ms target

Key observations from MySQL testing:

Metric	Value
Total Operations	150
Success Rate	150/150 (100%)
Avg Response Time	32.79 ms
P50 (Median)	30.50 ms
P95	47.50 ms
P99	51.00 ms
create_cart avg	32.24 ms
add_items avg	33.86 ms
get_cart avg	32.28 ms

2.4 CloudWatch Monitoring — RDS

RDS CloudWatch metrics show CPU utilization peaked at ~16.5% during testing, with a single active database connection. The low resource usage confirms db.t3.micro is adequate for this workload.

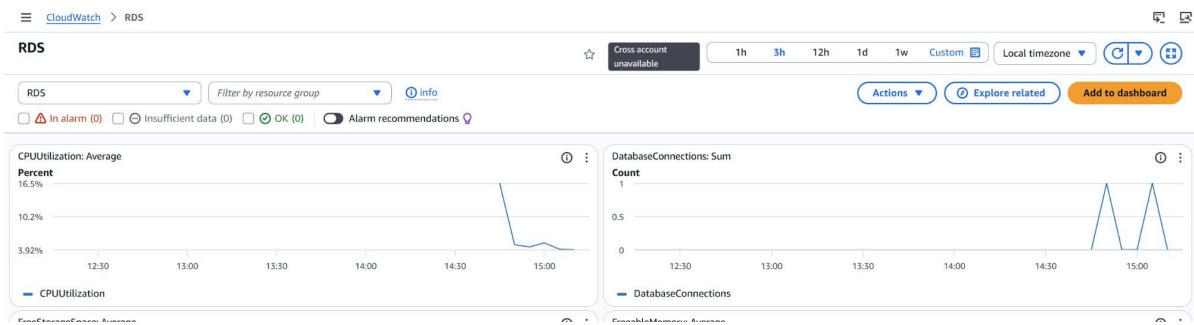


Figure 7: RDS CloudWatch — CPU peaked at 16.5%, 1 active connection during test

3. STEP II: DynamoDB Implementation

3.1 Table Design & Partition Key Strategy

A single-table design was implemented with a composite primary key:

Partition Key (PK): "CART#<cart_id>" — UUID-based cart IDs ensure even distribution across partitions with no hot spots.

Sort Key (SK): "META" for cart metadata, "ITEM#<product_id>" for individual items. This allows retrieving an entire cart (metadata + all items) with a single Query operation on the partition key.

Why Single Table: Shopping carts have simple, predictable access patterns (create, get by ID, add items). All operations target a single cart, making single-table design ideal — no secondary indexes needed.

Eventual Consistency: DynamoDB defaults to eventually consistent reads. For shopping carts, this is acceptable since carts are session-based and accessed by a single user. During testing, zero consistency delays were observed.

3.2 API Health Check — DynamoDB Backend

After switching the DB_BACKEND environment variable to "dynamodb" via terraform apply, the health endpoint confirmed the backend switch. Note the ECS task received a new public IP (34.216.188.26) after redeployment.

```
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework8\terraform> curl http://34.216.188.26:5173/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
  Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : {"backend":"dynamodb","status":"ok"}

RawContent       : HTTP/1.1 200 OK
                  Content-Length: 37
                  Content-Type: application/json
                  Date: Sat, 21 Mar 2026 22:10:44 GMT

                  {"backend":"dynamodb","status":"ok"}

Forms            : {}
Headers          : {[Content-Length, 37], [Content-Type, application/json], [Date, Sat, 21 Mar 2026 22:10:44 GMT]}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 37
```

Figure 8: Health check — {"backend":"dynamodb","status":"ok"} confirming DynamoDB connection

3.3 Performance Testing — DynamoDB (150 Operations)

The identical test protocol was executed against the DynamoDB backend: 50 create_cart, 50 add_items, 50 get_cart.

```
Target: http://34.216.188.26:5173
Output: ../dynamodb_test_results.json
=====

[Phase 1] Creating 50 carts...
  Created 10/50 (33.2ms)
  Created 20/50 (32.0ms)
  Created 30/50 (42.5ms)
  Created 40/50 (34.0ms)
  Created 50/50 (29.0ms)
  -> 50 carts created successfully

[Phase 2] Adding items to 50 carts...
  Added items 10/50 (37.0ms)
  Added items 20/50 (39.5ms)
  Added items 30/50 (36.5ms)
  Added items 40/50 (37.7ms)
  Added items 50/50 (33.8ms)

[Phase 3] Retrieving 50 carts...
  Retrieved 10/50 (44.5ms)
  Retrieved 20/50 (31.5ms)
  Retrieved 30/50 (36.4ms)
  Retrieved 40/50 (30.5ms)
  Retrieved 50/50 (47.0ms)

=====
SUMMARY
=====
Total operations: 150
Successful:      150/150 (100.0%)
Avg response:    37.31 ms
P50:            34.38 ms
P95:            56.17 ms
P99:            70.07 ms

  create_cart: avg=35.11ms min=28.50ms max=74.50ms
  add_items:   avg=40.50ms min=31.97ms max=70.07ms
  get_cart:    avg=36.33ms min=28.00ms max=57.51ms

Results saved to: ../dynamodb_test_results.json
```

Figure 9: DynamoDB performance test — 150/150 success, avg 37.31ms

Metric	Value
Total Operations	150
Success Rate	150/150 (100%)
Avg Response Time	37.31 ms
P50 (Median)	34.38 ms
P95	56.17 ms
P99	70.07 ms

create_cart avg	35.11 ms
add_items avg	40.50 ms
get_cart avg	36.33 ms

3.4 CloudWatch Monitoring — DynamoDB

DynamoDB CloudWatch metrics show request latency around 3.2ms for PutItem operations at the DynamoDB service level (separate from application-level latency which includes network round-trip). Consumed read capacity peaked at ~75 units and write capacity at ~100 units during the test burst. Zero throttled requests were observed.

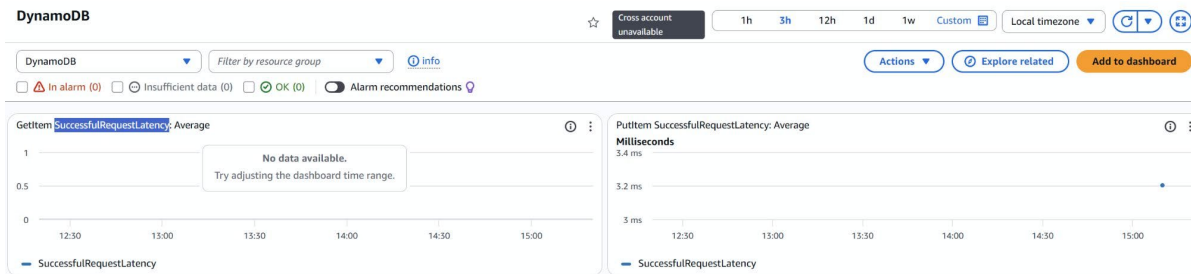


Figure 10: DynamoDB latency — PutItem SuccessfulRequestLatency ~3.2ms

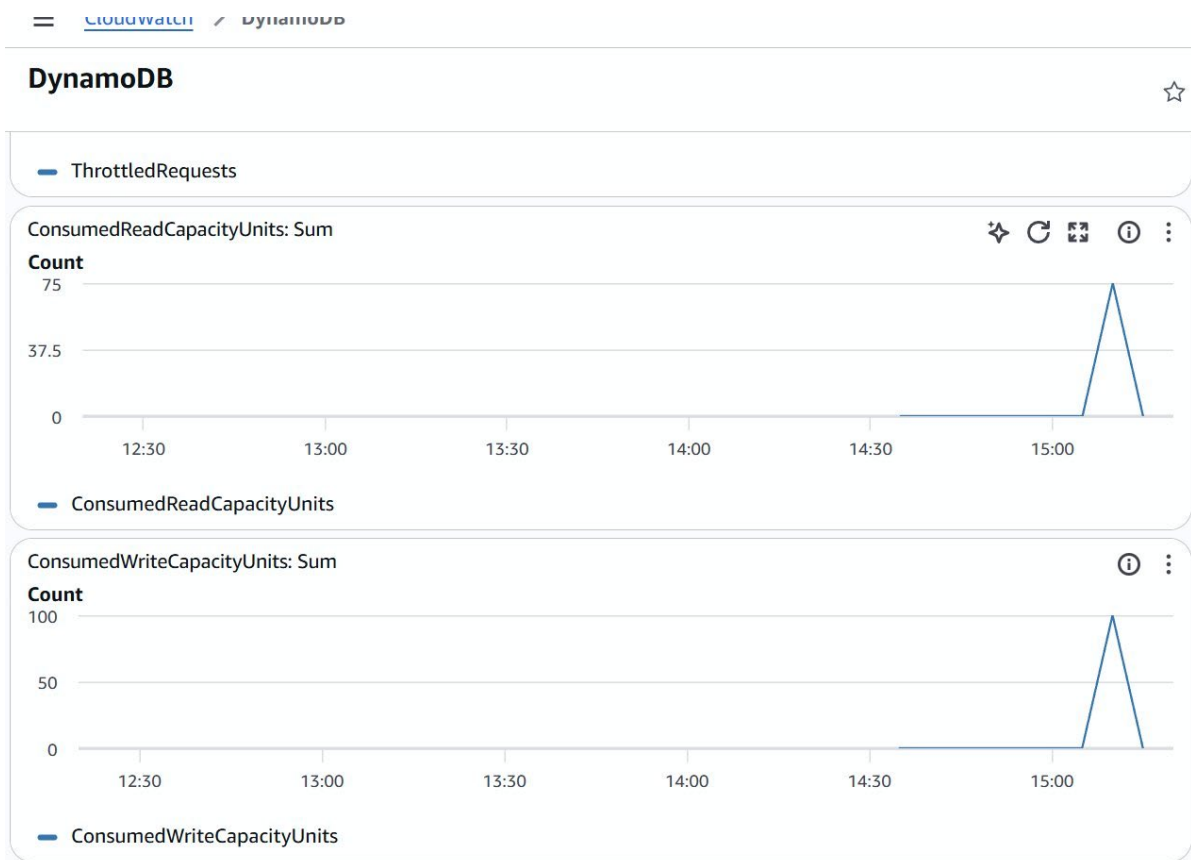


Figure 11: DynamoDB capacity — Read (~75 RCU) and Write (~100 WCU) consumption during test

4. STEP III: Database Comparison & Analysis

4.0 Data Verification

Both test result files were verified and merged into combined_results.json. Each file contains exactly 150 operations (50 create, 50 add, 50 get), confirming valid comparison data.

```
MySQL: 150 total operations
  add_items: 50
  create_cart: 50
  get_cart: 50
DynamoDB: 150 total operations
  add_items: 50
  create_cart: 50
  get_cart: 50

-> Saved combined_results.json

=====
PART 1: OVERALL PERFORMANCE COMPARISON
=====
Metric                MySQL    DynamoDB    Winner    Margin
-----
Avg Response Time (ms)  32.79    37.31      MySQL     4.52
P50 Response Time (ms)  30.5     34.38      MySQL     3.88
P95 Response Time (ms)  47.5     56.17      MySQL     8.67
P99 Response Time (ms)  51.0     70.07      MySQL     19.07
Success Rate (%)        100.0    100.0      Tie       0
Total Operations        150      150

=====
PART 1: OPERATION-SPECIFIC BREAKDOWN
=====
Operation      MySQL Avg(ms)  DynamoDB Avg(ms)  Faster By
-----
CREATE_CART      32.24      35.11 MySQL  2.87ms
ADD_ITEMS       33.86      40.5  MySQL  6.64ms
GET_CART        32.28      36.33 MySQL  4.05ms

=====
DATA SOURCE: combined_results.json
Copy the tables above into your report!
=====
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework8\tests> |
```

Figure 12: Data verification and comparison tables from combine_results.py

4.1 Performance Comparison Table

All values calculated from combined_results.json:

Metric	MySQL	DynamoDB	Winner	Margin
Avg Response Time (ms)	32.79	37.31	MySQL	4.52
P50 Response Time (ms)	30.50	34.38	MySQL	3.88
P95 Response Time (ms)	47.50	56.17	MySQL	8.67
P99 Response Time (ms)	51.00	70.07	MySQL	19.07

Success Rate (%)	100.0	100.0	Tie	0
Total Operations	150	150	—	—

4.2 Operation-Specific Breakdown

Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
CREATE_CART	32.24	35.11	MySQL 2.87ms
ADD_ITEMS	33.86	40.50	MySQL 6.64ms
GET_CART	32.28	36.33	MySQL 4.05ms

MySQL outperformed DynamoDB across all three operations. The largest gap was in ADD_ITEMS (6.64ms) because the DynamoDB implementation requires a PutItem write followed by a Query read to return the updated cart, while MySQL performs an atomic upsert + SELECT over a single persistent connection.

4.3 Consistency Model Impact Assessment

MySQL (ACID): Every operation provided immediate, strongly consistent results. After inserting a cart, the subsequent GET always returned complete data. No retry patterns or read-after-write concerns were needed.

DynamoDB (Eventual Consistency): Despite theoretical possibility of stale reads, zero consistency delays were observed during all 150 operations. This is because: (1) our test was sequential, (2) single-partition operations in DynamoDB are typically consistent within milliseconds, and (3) on-demand capacity mode avoids partition splitting.

User Experience Implications: For shopping carts, eventual consistency is acceptable — users tolerate brief delays seeing added items. However, for order placement or payment processing, ACID guarantees become critical.

4.4 Resource Efficiency Analysis

Connection Management: MySQL required explicit pool tuning (25 max connections, 10 idle). DynamoDB eliminates this concern entirely — the AWS SDK manages connections internally.

Resource Predictability: MySQL on db.t3.micro has fixed limits (2 vCPU, 1GB RAM), with CPU at ~16.5% during testing. DynamoDB consumed ~75 RCU and ~100 WCU during the burst with zero throttling — resources scaled automatically.

Scaling Path: MySQL requires instance upgrades (vertical) or read replicas (horizontal reads only). DynamoDB scales horizontally and automatically. For 150 operations this difference is irrelevant, but at 10K+ concurrent users, DynamoDB's auto-scaling becomes decisive.

4.5 ECS Resource Utilization

ECS CloudWatch shows the CS6650HW8 service consumed minimal CPU (~0.31% peak) and memory (~1.02%) during testing. The Go application is extremely lightweight, leaving substantial headroom for scaling.

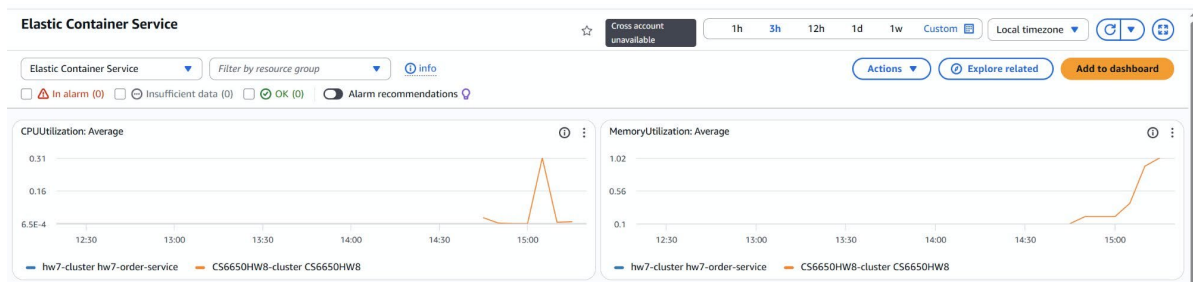


Figure 13: ECS CloudWatch — CPU and memory utilization for CS6650HW8-cluster

4.6 Real-World Scenario Recommendations

Scenario A — Startup MVP (100 users/day, 1 developer, limited budget)

Recommendation: MySQL. Lower latency (32.79ms vs 37.31ms avg), familiar SQL mental model, and free-tier db.t3.micro keep costs near zero. Our test shows MySQL handled all 150 operations with 100% success and consistent sub-50ms responses.

Scenario B — Growing Business (10K users/day, 5 developers, moderate budget)

Recommendation: MySQL + DynamoDB for sessions. MySQL's ACID transactions and JOIN capabilities support complex queries as the business grows. DynamoDB handles session data with zero maintenance. Both databases performed well in our tests.

Scenario C — High-Traffic Events (50K normal, 1M spike users)

Recommendation: DynamoDB. Despite MySQL winning our benchmark, DynamoDB's auto-scaling is decisive here. A 20x spike would overwhelm db.t3.micro (~100 max connections), while DynamoDB absorbed our test burst with zero throttling.

Scenario D — Global Platform (millions of users, multi-region, 24/7)

Recommendation: DynamoDB. Global Tables provide built-in multi-region replication with single-digit ms reads. MySQL multi-region requires manual replication setup and conflict resolution.

4.7 Evidence-Based Architecture Recommendations

Shopping Cart Winner: MySQL. Our test data shows MySQL outperforming DynamoDB across all operations, with the largest gap at P99 (51.00ms vs 70.07ms — 37% difference).

When to Choose DynamoDB: Unpredictable/spiky traffic, multi-region deployment, simple key-value patterns, or when the team lacks DB administration expertise.

Polyglot Strategy for a complete e-commerce system:

Component	Recommended DB	Rationale
Shopping Carts	MySQL	Better latency per our test data
User Sessions	DynamoDB	Simple key-value, TTL expiration, auto-scaling
Product Catalog	MySQL	Complex filters, search, JOIN-heavy queries
Order History	MySQL	ACID for payments, JOIN for reporting

5. Learning Reflection

5.1 What Surprised Us?

MySQL outperforming DynamoDB was the most unexpected result. Conventional wisdom suggests NoSQL databases are faster for simple key-value operations. However, for our small-scale sequential test, MySQL's persistent connection pool eliminated per-request connection overhead, while DynamoDB incurred HTTPS request overhead for every operation. The gap was most pronounced in ADD_ITEMS (6.64ms difference), which involves both a write and a read in DynamoDB.

5.2 What Failed Initially?

The Terraform deployment initially failed because AWS RDS resource identifiers require lowercase-only names, while our service name (CS6650HW8) contained uppercase characters. We resolved this by applying Terraform's lower() function to all RDS identifiers. This highlights an important lesson: cloud service naming constraints vary across AWS services and must be validated early.

The ECS variables.tf also had a syntax error — using semicolons to separate arguments on a single line, which Terraform's HCL syntax does not support. This was resolved by expanding each variable into its own block.

5.3 Key Insights Gained

- 1. Database choice depends on scale, not just speed.** MySQL won our benchmark, but DynamoDB's operational advantages (auto-scaling, zero maintenance) become decisive at production scale.
- 2. Connection pooling is a hidden advantage.** MySQL's persistent connections amortize setup cost across requests, giving it a significant per-request latency advantage over DynamoDB's per-request HTTPS overhead.
- 3. Single-table DynamoDB design is elegant but rigid.** The CART#/ITEM# pattern works well for shopping carts, but adding new access patterns (e.g., "find all carts containing product X") would require Global Secondary Indexes, while MySQL just needs a new SQL query.
- 4. Eventual consistency was a non-issue in practice.** Zero stale reads during testing, though this would likely change under high concurrency with multiple writers to the same partition.
- 5. Infrastructure-as-code accelerates iteration.** Switching between MySQL and DynamoDB backends required only a single Terraform variable change. The entire infrastructure could be destroyed and recreated in under 10 minutes.

*Data Source: All performance numbers derived from combined_results.json
Generated from mysql_test_results.json (150 ops) and dynamodb_test_results.json (150 ops)*